

System Requirements Documentation

Coffee Shop Management System

Adriel Nguyen

16 December 2025

Table of Contents

1. Customer Problem Statements and System Requirements
2. Functional Requirement Specification
3. System sequence diagram
4. Activity diagram
5. User interface specification
6. Traceability matrix
7. System architecture and system design
8. Algorithms and data structures
9. User interface design and implementation, design of tests
10. Project plan
11. Reference

References:

[<https://unsplash.com/>]

- <https://images.unsplash.com/photo-1517640033243-dc06bb716df5?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=687>
- <https://images.unsplash.com/photo-1658646479124-bc31e6849497?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=1251>
- <https://images.unsplash.com/photo-1670515854747-3d9be05927b9?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=1964>
- https://plus.unsplash.com/premium_photo-1661370085023-16bf34456bbf?ixlib=rb4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=687
- <https://images.unsplash.com/photo-1527678357412-ef45dfbd9ecc?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=687>
- <https://images.unsplash.com/photo-1674642387251-04a5f9320aa4?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=687>
- <https://images.unsplash.com/photo-1645592241237-3b05af6c194f?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=687>
- <https://images.unsplash.com/photo-1527156231393-7023794f363c?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=711>
- <https://images.unsplash.com/photo-1548839140-29a749e1cf4d?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=688>
- https://plus.unsplash.com/premium_photo-1671660015357-112b00365d77?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&q=80&w=687
- <https://images.unsplash.com/photo-1617253426941-d99b541c18fe?ixlib=rb-4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D>

[&auto=format&fit=crop&q=80&w=687](#) ByteByteGo -- knowledge, how system design works, and valuable information. Super amazing reliable source

- <https://www.youtube.com/@ByteByteGo/posts>

Coffee shop management system Adriel Nguyen

Problem Statement:

Coffee shops are ready to open in the morning, but 30 customers need to wait in line for purchases.

Managing a coffee shop is not easy, from closely keeping track of supplies bought, customer progress, and employees' sales. The biggest problem that impacts the company is when a manager does not know when to order the types of supplies needed for employees to make drinks for the customers.

Improper coffee shop management directly leads to consistent customer complaints. Aside from this, it prevents opportunities for the customer to purchase the service in terms of ending up wasting their time, they do not know that such customer benefits exist, or perhaps their manager cannot see how much employees' sales increase during their working hours.

Objectives of the system:

The purpose of the system helps the coffee shop serve coffee strategically to the customer under time constraints, which would accelerate faster flow, more productivity, and easier control. For instance, it would let customers preorder a menu when waiting, split into lanes, and waiting numbers to call; all these three activities are happening at the same time. Users can keep track of their membership progress, which would let them know when their drinks are ready to pick up.

The systems will help coffee managers to visualize current inventory and to order ingredients accordingly.

Typical customers

Office workers, freelancers, researchers, students, real estate agents, ...

System Requirements:

- Managers can plan to order the necessary ingredients/materials.
- Employees can keep track of user information, their orders, and exchange membership points to buy drinks, set up a pre-order process, activate self-service orders, and manage waiting numbers.
- Customers can view access membership, order drinks, and maybe use a bar code to pay for the drink.

Project planning and development

1. Software:

Front-end: Node.js, CSS

Back end: Python, Java

Database: MySQL / MariaDB

Framework use: SymPy, NumPy, Apache NetBeans

1. Hardware: Personal laptop

2. No additional plugins needed for networks

Week 1: Writing pseudocode, gathering information, and reading books relating to data structure, database, and preparing presentations properly. GitHub commit and to-do lists.

Week 2: Make sure backend and frontend can communicate with each other; the backend can update and change in real time

Week 3: Customer progress register, track supplies mode, employee sales record,

Week 4: Adds on other functionality, preordering, self-order lanes, and ticket number

Week 5: GUI and interaction, add-ons and code revision, update and record the demo.

Week 6: Run multiple live tests, fix mistakes

Week 7: Test edge cases and note abnormal changes

Week 8: Record final demo and presentation

Problem Statement:

Users often find it takes longer time to wait for their turn when they need to buy the drinks they want in busy hours. One of the things that make us feel uncomfortable is trying to reduce the time we must wait. We hope there is a system analyzer who comes up with a solution that can address this issue

Coffee shops are ready to open in the morning, but 30 customers need to wait in line for purchases. Managing a coffee shop is not easy, from closely keeping track of supplies bought by customer progress, and employees' sales. A coffee shop management system is always identifiable in many businesses that work with food and beverage industries. There are two slots, one slot role is act as a direct payment, users can pay directly through cash, or they can use credit card to purchase the beverages and food they want. There is also the second slot, which is preorder section, where coffee is ordered automatically through a system from user interaction, it then prints out the receipts. Users receive a number ticket to wait until their turn to get the drinks they bought. Additionally, users can pay upfront using barcode and be able to see their membership points in the user profile. The biggest problem that impacts on the company is when a manager does not know when to order the types of supplies needed for employees to make drinks for the customers. Aside from this, it prevents opportunities for the customer to purchase the service in terms of ending up wasting their time, they do not know that such customer benefits exist, or perhaps their manager cannot see how much employees' sales increase during their working hours.

Glossary of Terms: Define important terms to make them consistent in the system specifications.

Terms list:

Preordering – is when user is can order before they have to move on to wait on the queue line

Menu – it lets the user to decide what makes sense the most to them and simply select on what they see

App server – is when pulling out the necessary information we need from a database, basically like extracting the user information and then use it for further processing calculation. For example, user in database has 100 membership points, when user reaching upon 109 membership points, they will receive a 40% reduction in the next purchase

Form filling – in preordering settings, there is a feature which guides and support the user to interact with the beverage and write their opinions about the food, drinks think of it like a brief review of what the user feel after trying the products. This helps to improve the service through getting the most out of customer feedback.

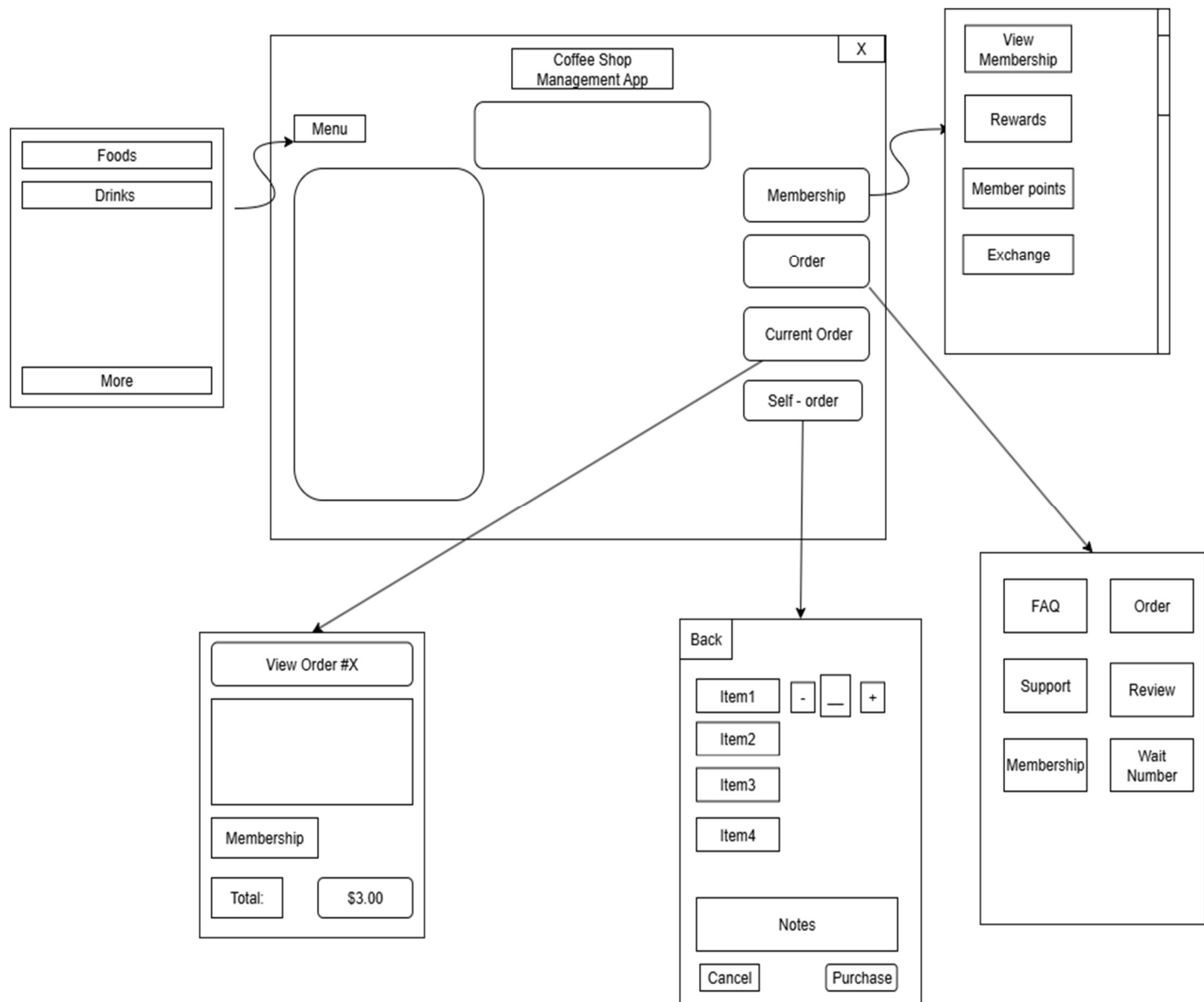
System Requirements

Functional Requirements

No.	Priority Weight	Description	Non function Requirements	Description of nonfunctional requirements
REQ-1	High	Membership Points features, allow users to use the points to buy drinks or convert them to vouchers for next time use	Functional	The features should be compatible with other features; it would do the right job that have been assign to it
REQ-2	High	Activates self service to let user automatically order, obtain receipts to transfer on ticket wait	Supportability	High customization allows the user to add on many things as they would love to add notes on the ingredient's percentages For example: customers want more sweet, less sweet or more ice, less ice.
REQ-3	High	Payment features, Let user purchase drinks and food, print out a receipt	Reliability	The system should have high accuracy and some how be predictable toward display the receipts to serve prints out purpose as evidence of customers purchase the drinks
REQ-4	High	Allows managers to view / adjust sales record, track ingredients and managing customer information	Usability	Must be responsive to use and easy to explain to other employees how to use the system through training
REQ-5	Medium	The system should be handling for 30 customers' requests simultaneously and no freezing the GUI	Performance	This is the steps that needs of carefully design since once the app is not responsive, everything have to return back to start.
REQ-6	Medium	If user willfully wants to quit or go back to the previous steps or change their choices,	Functionality	Modular, each part has buttons to do the job of go back to the main menu,

		the system must allow the user to do that		adding selections and remove selections
REQ-7	Low	Customers should feel the system is easy to use, manually simple to understand and easy to interact with	Reliability	Block size, with simple GUI design, and working mathematical operations, all buttons should work, like user expect a button to open, it should be open instead of closing the app.
REQ-8	Low	The coffee management system should not show or add complex operations and formulas shown on the client side which makes them feel it too technical and complex	Supportability	Remove complex formulas and hard coded in the program instead of leaving out to let the user see it

User Interface Sketch Design



Plan of Work

Progress Report: 5%,

I have installed some dependencies and start working on the database to get ready for it to work, next I begin with incorporates the GUI and the Database works along with python. My goal is to finish the working functions, features and the app should expect to work (can handle at least 50 different test cases coverages) before Monday the 20th October.

Coffee Shop Website Information System

System Architecture and System Design

1. An architecture style that I use to design my coffee shop website is the component system. My thought process is breaking these big tasks into smaller components that I can reusable, I can easily replace what I've been implementing previously, for example, instead of having to create extra button what I will do is bring back the button feature I have already code and then update its logic. Another thing I have observed is I specifically customize the website in this way, rather than letting the user have to manually switch in between different .html file like manager dashboard and employee and then the customer view, I decide to wrap that component to those specific kinds of user

Manager -- has manager role

Employee -- has employee role

Customer -- has customer role

Role assign makes things clear for those components that are design to shows and specific commands this user can access to.

2. The server connects and communicates with the client through HTTP (hyper text transfer protocol locally) what it does is the front end sends to backend port and receives the information they need to send back from the back end

3. My server is host using python programming specifically I use fastAPI, build using uvicorn , and then in front end I use axios to bridge the connection. While design the system, one thing I love about using fastAPI is it very fast and the cool part of using uvicorn is whenever I make changes it reloads the server in real time (run via commandline)

4. I use MariaDb to store important user information

Network type:	MariaDB or MySQL (TCP/IP) ▼
Library:	libmariadb.dll ▼
Hostname / IP:	127.0.0.1
	☐ Prompt for credentials
	☐ Use Windows authentication
User:	root
Password:	
Port:	3306 ▲▼
	☐ Compressed client/server protocol
Databases:	Separated by semicolon ▼
Comment:	

Below are the tables I have created: KiB denotes Kilobytes

item table is used for storing coffee items

notification table is used for notification

orderitem is used for tracking order id, item id and the how much item selected

orders keep track of each transactions

user is used for tracking members

▼ coffee_shop_db 176.0 KiB	
item	16.0 KiB
notification	32.0 KiB
orderitem	48.0 KiB
orders	48.0 KiB
user	32.0 KiB

Three essential tables

notification table

Whos the sender ?

What message are they sending ?

Are they read the notification or not ? (to track they read or not tracks two things : are they online or not and two the time when the notification appears and when it close by user)
special detail : if user have not read it , shows a badge (font white , red background with number shows how many notifications are there, they can tap to view the message)

Yellow key: Primary key – unique

Green key: Foreign key userID is reference from the user table (below)

	#	Name	Datatype	Length/Set	Unsigned	Allow N...	Zerofill	Default
	1	id	INT	11	☐	☐	☐	No default
	2	userID	INT	11	☐	☐	☐	No default
	3	message	VARCHAR	255	☐	☐	☐	No default
	4	read	CHAR	50	☐	☐	☐	No default
	5	created_at	DATETIME		☐	☐	☐	current_timestam...

User table

Ah we got a red key, this is unique, this means each person email should be different

I use created_at with date time to time stamp (to debug and log in system)

	#	Name	Datatype	Length/Set	Unsigned	Allow N...	Zerofill	Default	Comment
	1	id	INT	11	☐	☐	☐	AUTO_INCREME...	
	2	name	VARCHAR	255	☐	☐	☐	No default	
	3	email	VARCHAR	255	☐	☐	☐	No default	
	4	password	VARCHAR	255	☐	☐	☐	No default	
	5	role	VARCHAR	50	☐	☐	☐	'customer'	
	6	created_at	DATETIME		☐	☐	☐	No default	

Here is what the user table store (database)

#	1 id 🔑	2 name	3 email 🔑	4 password	5 role ▲	6 created_at ▲
1	19	Olzhas Alton	olzhas@gmail.com	12345	customer	2025-10-24 16:29:37
2	20	Erik Alvin	erik@gmail.com	112233	customer	2025-10-24 16:29:37
3	21	Rahmesha Wilson	rahmesha@gmail.com	12345678	customer	2025-10-24 16:29:37
4	31	Adriel Handsome	adrielnguyenminh@gmail.com	12345	customer	2025-10-28 03:43:52
5	32	test	test@gmail.com	123	customer	2025-11-02 11:51:40
6	33	scoobydoo	scoobydoo@gmail.com	123	customer	2025-11-02 12:03:02

Here you can see all id are unique

Name from 1 – 3 are seeded

4 to 6 use for test case

So what does the database responsible for? It is a back bone of this project, where it store

1. User membership points
2. User personal information (real life it will be privacy information)
3. Bank details – cvv, 16 numbers on the card, card holder name

This application runs locally on one machine since it does not host using cloud service.

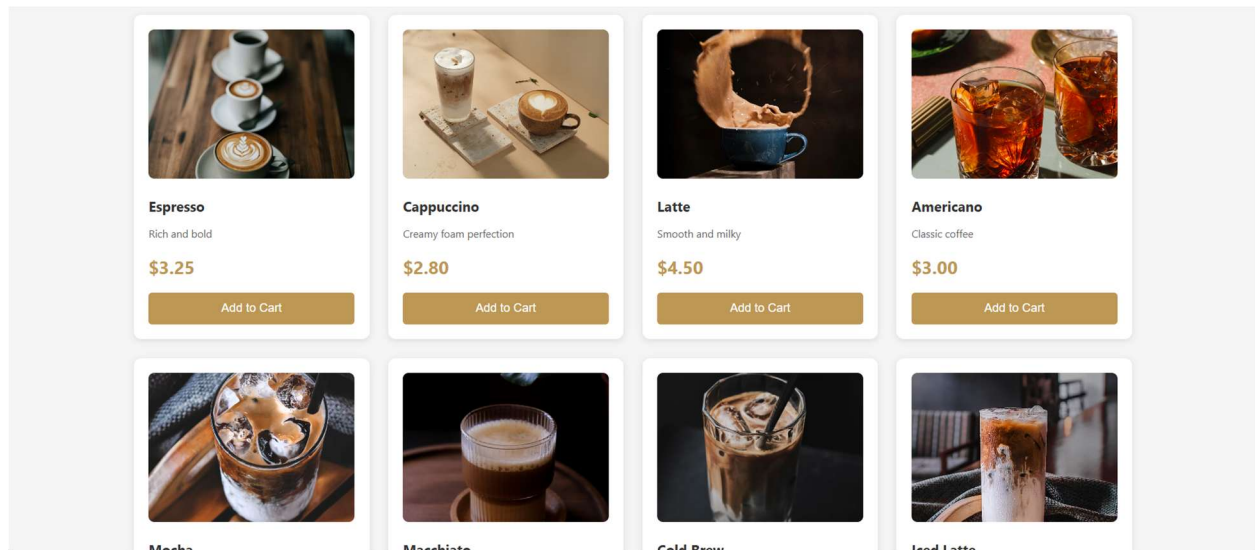
7. What does python play a role as a backend service in between database and the frontend? Python acts as an endpoint of the API get ways that use fastAPI with CRUD operations to transfer client front end interaction to process information from the database.

8. Where is the UI? At first, halfway through the project, I suddenly noticed from the mcd I couldn't install React from vite because I'm trying to use shadcnUI components, which is very nice. So, I decided to use html, css and java script. Main features are render through javascript, css , python, fastAPI and database. Html keeps minimal

Identify Subsystems

Discussion: At first glance, I thought what if I store user information on front end local storage, this in fact is a cool idea, but turns out to be very dangerous and vulnerable.

An illustration from my front-end UI menu (user, employee, and manager can see)
(note: image are references and attributes in a reference note, in the project main
readme.md images are copyright free)

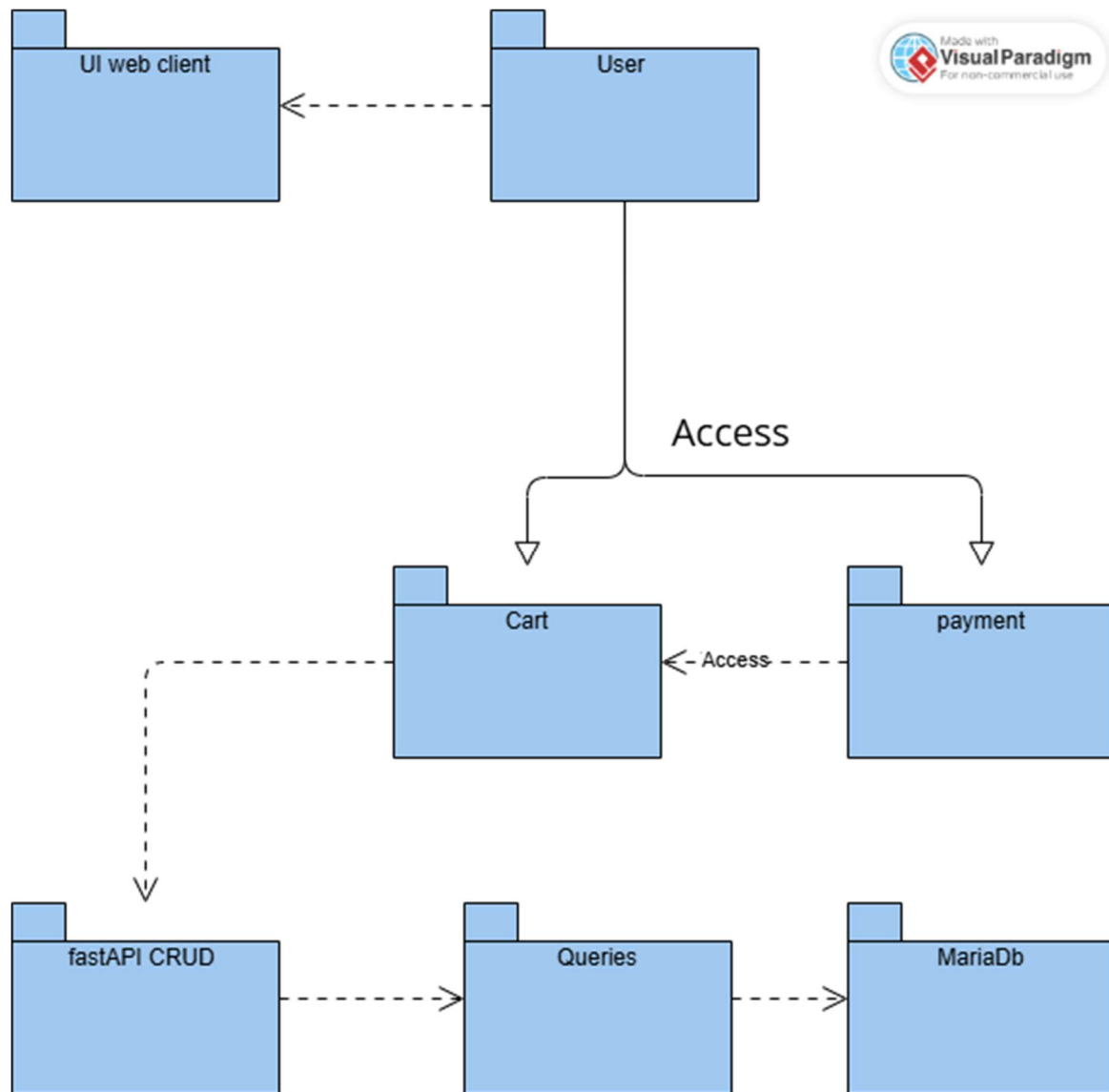


Html is use to gather user information, user type into the form label, it then validates first part by the javascript to ensure that everything is pass through first layer (8 characters, has @ in email things like that, then the information transfer by the system to the second layer which is python, it then use a class of model data, these are the user class which acts as object relation model that convert the user information into something the database can store, update , retrieves and delete through CRUD operations. The purpose is it makes life easier and efficient, how? instead of manually writing on a paper, now we use queries to makes task easier and store things digitally. A simple query can get the right information quickly within 0.12s and it is very detailed.

Add items to menu, update item from client front end to back end (real time)

SQL query is using a parameterized, which in mariadb this is very useful and a strong resistance against cross site scripting.

UML



Database Storage Layer

Data storage

What to store specifically,

1. Store dynamic information, for example how much does the user pay for an item (a drink or cake)
May be today drink cost: \$1.25 but tomorrow its \$1.5. Database management system can do this efficiently
2. Credit card: information can be store for returned purchases, what if the user needs to buy the item again, if without MariaDB, it will have to store somewhere in the back end, not secure and fast queries
3. Memberships and barcode – how much points do user have now and what's the barcode that represents the transaction that the user buy

It needs to be in a relational database with integrity check, in one of the databases, I must set the validation of quantity cannot be less than 0 because how can we order -1 items? so this is an example of one of the checks (others are referential, null check, integrity check)

Execution Orders

User interacts with the UI components at the front end, the system reacts by showing notifications, pop up dialog to ask for submit information, it then send to backend. How it glues together is when for example, if a user has tried to access let say gain access into the membership and user account without login, the fastAPI gateway and the server acts as not letting unauthorized individuals enter the frontend unless they register and login. This kind of action gives the user the freedom and ability to interact with the data visually, like primary and secondary color and gain access to role base thing they can do, like customer can only permit to view the menu, they are not allowed to have modifications on it.

What kind of system?

Time Dependency

Has time – but it displays from back end, the front end fetches as json and then displays the information (display local time, all another time stamps are track separately in database)

Major things to work on are fixing bugs and simplifying architecture (I'm quite over engineer it – it does work by connecting front end to backend, can send notification, can have role base login, can create , update, delete and get information from back end send and use by front end or backend – python)

Ah one more thing the front end have to access CORS to be able to connect to the backend, other wise it will not listen to the front end at different port.

Hardware requirements

Operating system: Window / Linux / MacOS (as long as can install dependencies to local computer)

Color: minimum 1366 x 768 or full HD 1080pixel

Computer: Home laptop

Memory: 2 Gb Ram

Hard Drive needs around 1 Gb Ram

Sequence Diagram

Actor: Employee

Object: Coffee shop GUI interface,

Payments

Steps of coffee payment:

1. Customer waits in line
2. Employees ask what drinks the customer may want
3. Employee inputs the drinks / cake and its quantity according to customer need and then the system calculates the amount of money the customer needs to purchase.
4. Customer gives money to pay for their order. Next, the employee gives the money back (show from the system) with receipts
5. If the employee sees the customer cannot complete the bar code scanning, the employee then will note the customer case with a description, or cancel the bar code, or choose another way to transfer (transfer through the employee account – the employee will be responsible for this).

Notes: when credit card is applied, if customer account balance amount not enough to purchase the drinks / cake. It will send out a display message to let the employee know.

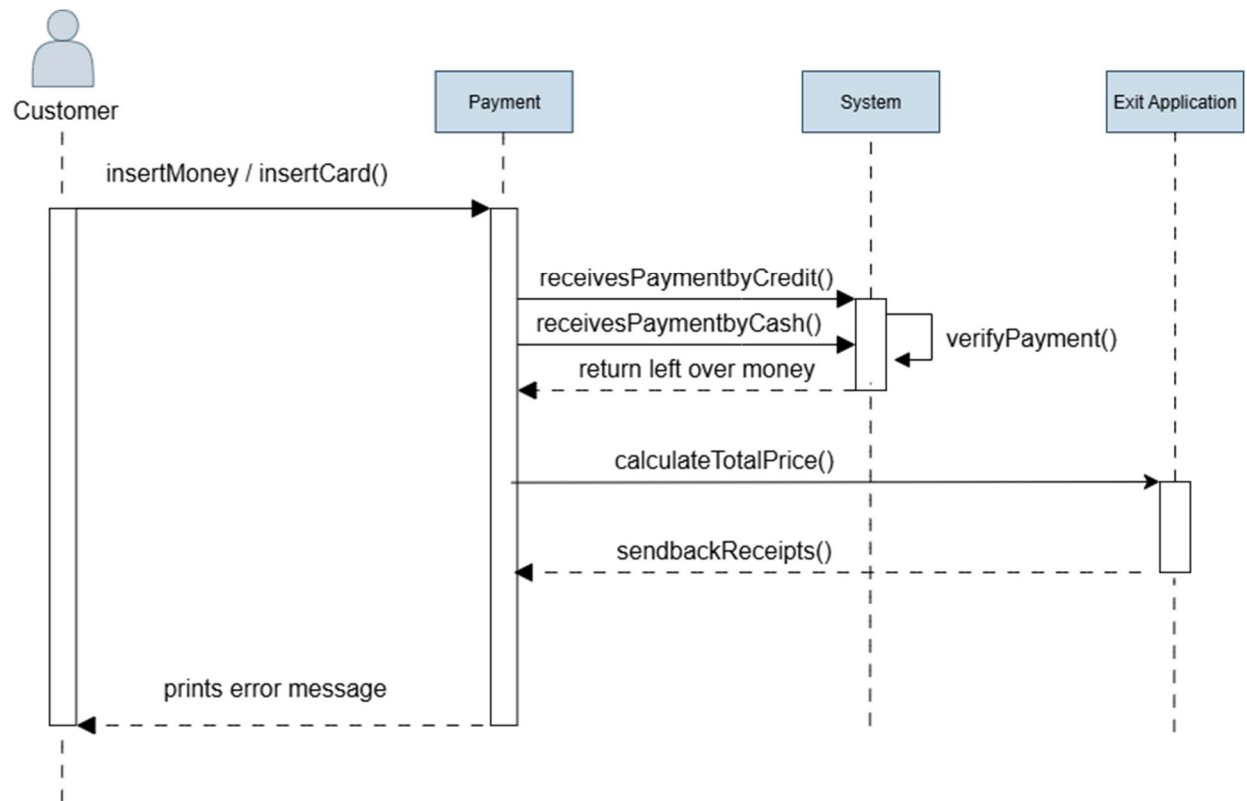
Self – Ordering (Order with no employee)

Actor: Customer

Object: Payment

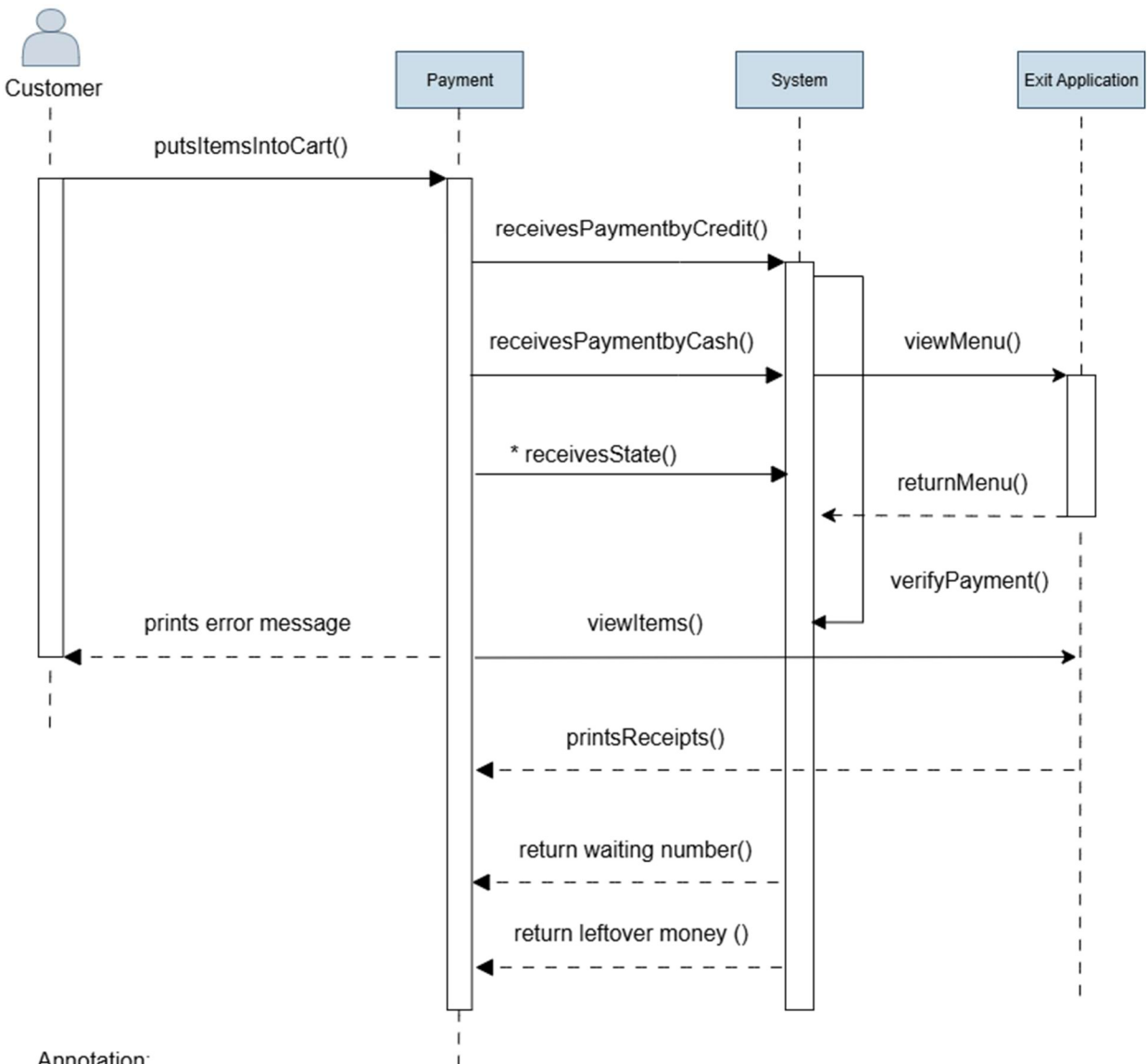
The sequence diagram:

1. The customer interacts with the system to select drinks
2. The systems shown checkout cart with items the customer has picked
3. The system will add up the total price and the user will see if it is correct
4. Display bar code / ask customer to put cash in (It will show the amount here)
5. When payment is gone through, customer receives the bill as print out format, if payment is invalid, customer would see errors according to follow reasons (card is unreadable, invalid balance)



Self-ordering

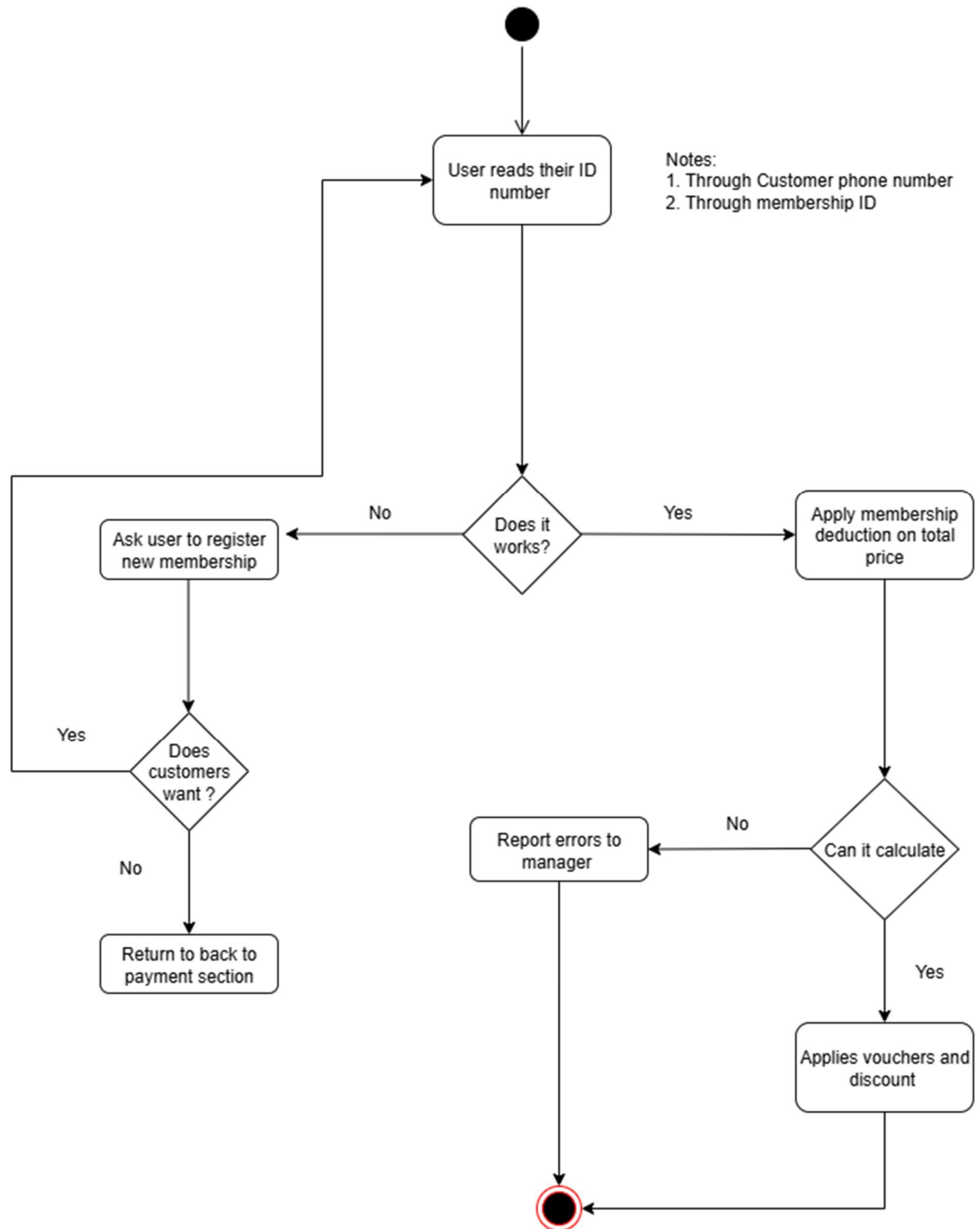
Self Ordering



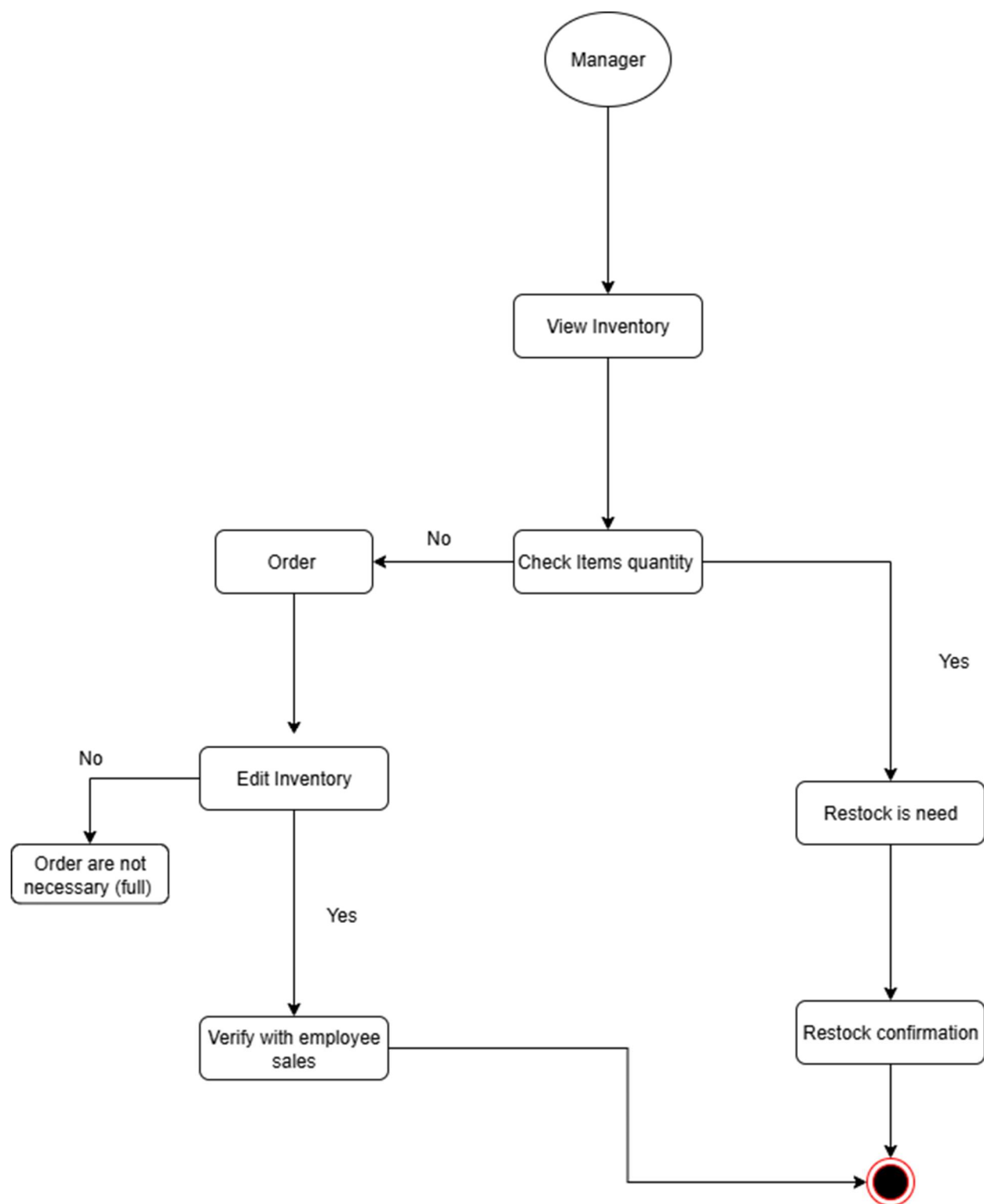
Annotation:

*Take away or sit in shop

Activity Diagram for Membership Use Case



Activity Diagram for Manager Inventory Use Case



Coffee Shop Management System

Preliminary Design

Use Case: Managers can view activities

Manager Front View



The diagram illustrates the Manager Front View interface. It features a light blue background with a central white table and various control buttons. At the top, there is a 'Back' button, a title box 'Coffee Shop Management System', and a 'Date Time' box. The table has three columns: 'Item ID', 'Item Quantities', and 'Item Prices'. To the right of the table is a vertical stack of green buttons: 'edit', 'add', 'find', 'delete', 'import', and 'export'. Below these is a 'Note:' label and a large white text area. At the bottom, there is a green 'save' button, a red 'CANCEL' button, and a green 'APPROVE' button.

Item ID	Item Quantities	Item Prices

Buttons: Back, Coffee Shop Management System, Date Time, edit, add, find, delete, import, export, Note:, save, CANCEL, APPROVE



Back

Coffee Shop Management System

Date Time

Item ID	Item Quantities	Item Prices

edit

add

find

delete

import

export

Note:

Manager has successfully added Items into the inventory

CANCEL

APPROVE

Use Case: Employee view receipts prints out and payment checking

Employee Front View

Coffee Shop Management System

Date Time

Employee Login

Username

Password:

Log in

Receipt Front View

Barcode	Date time	#Order Number
	Employee Name	
	Instore	

	1. Item name	\$2.50
2. Item name	\$2.50	
3. Item name	\$2.50	

Total:	\$7.50	
Change:	\$2.50	

Earn membership points everytime purchase

Use Case: Self order & Pre order

Coffee Shop Management System

Date time

In-store

Take-away

BACK

Coffee Shop Management System

Date time

Menu

Drinks

Item 1

Item 2

Item 3

Item 4

Pastry

Item 1

Item 2

Item 3

Item 4

Date time

#Order Number

Customer ID

Instore

1. Item name

\$2.50

2. Item name

\$2.50

3. Item name

\$2.50

Total:

\$7.50

Change:

\$2.50

☐ Take-away

Pay

Use Case: Membership points

BACK

Customer Membership Information

Date time

Current Points
200

Next Reward
300 points

Next Reward
400 points

Redeem
History

Exchange
points

Events

Customer ID

BACK

Redeem Membership Points

Date time

Points: 200

Points need:
100 points

5% discount

FALL2025

Points need:
140 points

7% discount

HAPPYDEAL2025

Points need:
350 points

12.5% discount

COFFEESHOP125

Redeem

Customer ID

BACK

Redeem Membership Points

Date time

Points: 200

Points need: 100 points	5% discount	FALL2025
Points need: 140 points	7% discount	HAPPYDEAL2025
Points need: 350 points	12.5% discount	COFFEESHOP125

Redeem

You have been successfully redeemed points !

Customer ID

BACK

Redeem Membership Points

Date time

Points: 200

Points need: 100 points	5% discount	FALL2025
Points need: 140 points	7% discount	HAPPYDEAL2025
Points need: 350 points	12.5% discount	COFFEESHOP125

Redeem

Error ! This redeem code has been used.
Insufficient points

Customer ID

Use Case: Track and manage user information

Manage User Information			Date time
CustomerID	EmployeeID	ProductID	Find Filter Save Changes
			Edit Add Delete Retrieve
Customer ID	Log out		

User Effort Estimation

Usage	Navigation	Clicks	Keystroke
View activities	Home, Employee Login, View activities	3	< 9
View receipts	Home, Employee Login, View receipts	3	< 7

Self-Order & Preorder	Home, instore, takeaway, Select items, check box, pay	6	< 8
Membership points	Home, back, redeem history, exchange point, events, voucher name, redeem	8	< 10
Track and manage user information	Home, back, find, filter, save change, edit, add, retrieve, log out	9	< 12

Extra information

Employee Front View

Coffee Shop Management System

Date Time

Username:

Password:

Log in

Version 1.0

Employee - Front View

Back

Now Ordering #X

Date Time

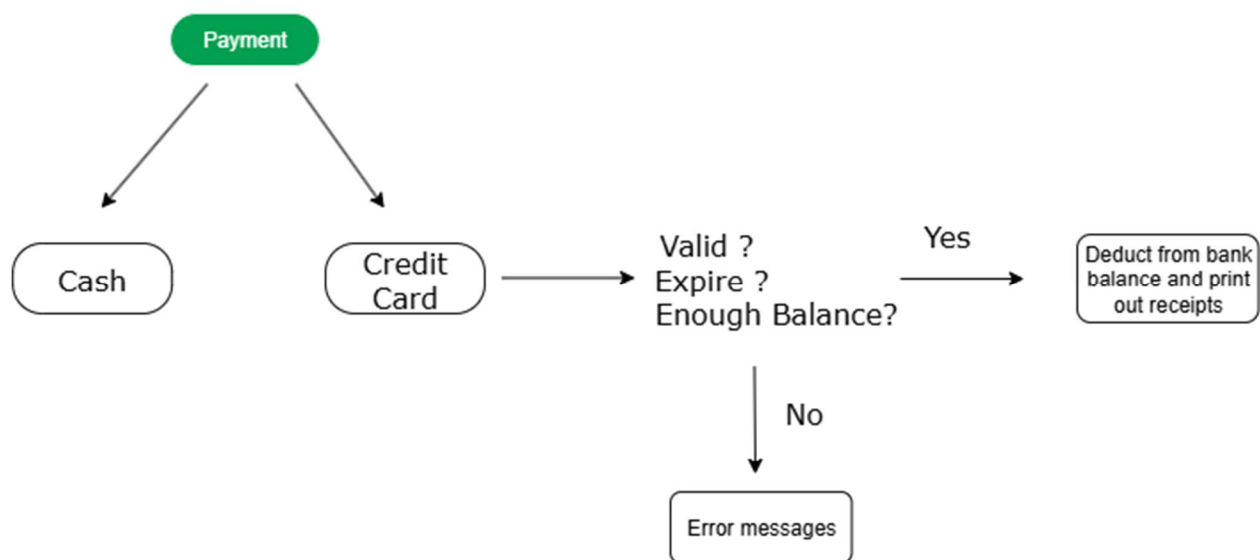
Order History

New

Name	Qt	Price
Prdct ID	1	\$2.50

☐ Applies Voucher

Total:



Functional Requirement Specification

Stakes holders:

Primary actors:

- Customer: This actor can participate in pre-order coffee, self-order coffee, wait in line, pay for the drink they need and wait to call for the number to pick up their drink.
- Sponsor: This actor can participate in funding the money for open coffee operation, their interest upon the project is how to make sure it is running efficiently
- Designer: This actor can participate in helping design the design to help improve the user interface and user experience to make the app looks good.

Secondary actors:

- Employees: This actor can interact with the customer guide and assign the user through the process of ordering coffee from start to finish, help user register for membership
- Managers: This actor can view reports, can check inventory, and have higher roles than employees
- System: giving details about drinks, inventory availability, user membership points, and drinks information, total income daily

Manager (12)

- Tracking: Manager can view activities (3)
- Adjust sales records (3)
- Track Ingredients (2)
- Manager user info (2)
- Managers must check if employees are correctly converting vouchers or not (2)

Employee (11)

- Log in / Log out: From employee account. Employees can login with their own username and password (2)
- View account: Let employees view their account (2)
- Scan bar code: Let employees scan bar code (represent and simulate payment through credit card) (3)

- Pay by cash: Let the user give back changes and total (2)
- Print receipt: print out the receipt on amount that customers bought from the store (1)
- Help users set up membership accounts (1)

Customer (11)

- Preordering: To order before they come or user can wait and order when other people are in line (2)
- Wait in line: Directly interact with employees to order drinks (3)
- Self ordering: Directly interact with the kiosk system, order the drinks they need (3)
- Pay for the drink: Customers are waiting in line and Self ordering must pay for the drink (2)
- Wait for the call number: Customer sits and waits until their number is being called on the ticket receipt and shows. (1)

System (13)

- Membership point: calculates the point and allow user to use this point to buy drinks and use it as discount for next drinks (3)
- Self – Service ordering: User will be able to order for take away and order without employee (2)
- Payment feature: let user purchase drinks and food, print out a receipt (1)
- Back features selection, let user to go back to where they want to go (1)
- Scan bar code to pay (2)
- Display the GUI, Display menu selection, (3)
- Let user interact with the system, show drinks, food with its prices (1)

```
graph LR
    Manager((Manager))
    Employee((Employee))
    Customer((Customer))
    System((System))

    Manager --- UC1((View Activities))
    Manager --- UC2((Adjust Sales Records))
    Manager --- UC3((Track Ingredients))
    Manager --- UC4((Manage User Info))
    Manager --- UC5((Preordering))
    Manager --- UC6((Self Ordering))
    Manager --- UC7((Wait in line))
    Manager --- UC8((Cash))
    Manager --- UC9((Wait for call))
    Manager --- UC10((Pay for Drink))

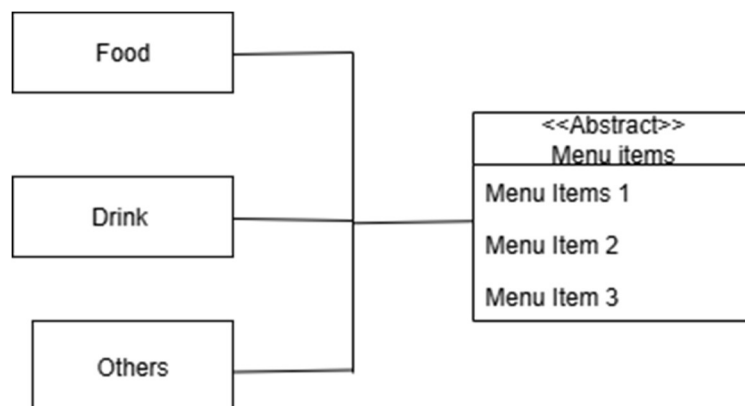
    Employee --- UC11((Print Receipt))
    Employee --- UC12((Scan BarCode))
    Employee --- UC13((Login / Logout))
    Employee --- UC14((Pay Cash))
    Employee --- UC15((View Account))

    Customer --- UC16((Membership Point))
    Customer --- UC17((auto service))
    Customer --- UC18((Scan Bar Code))
    Customer --- UC19((Check Voucher validity))
    Customer --- UC20((Register Membership))

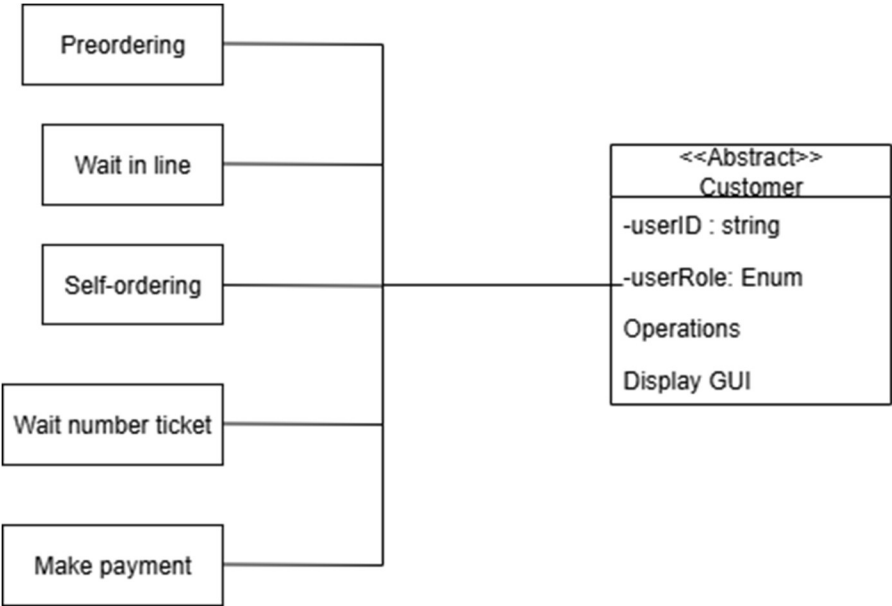
    System --- UC21((Add Food / Drinks & Prices))
    System --- UC22((Show Drinks & Foods))
    System --- UC23((Provides payment features))
    System --- UC24((Display Interaction))
```

Scan Bar Code <<include>> Pay for Drink

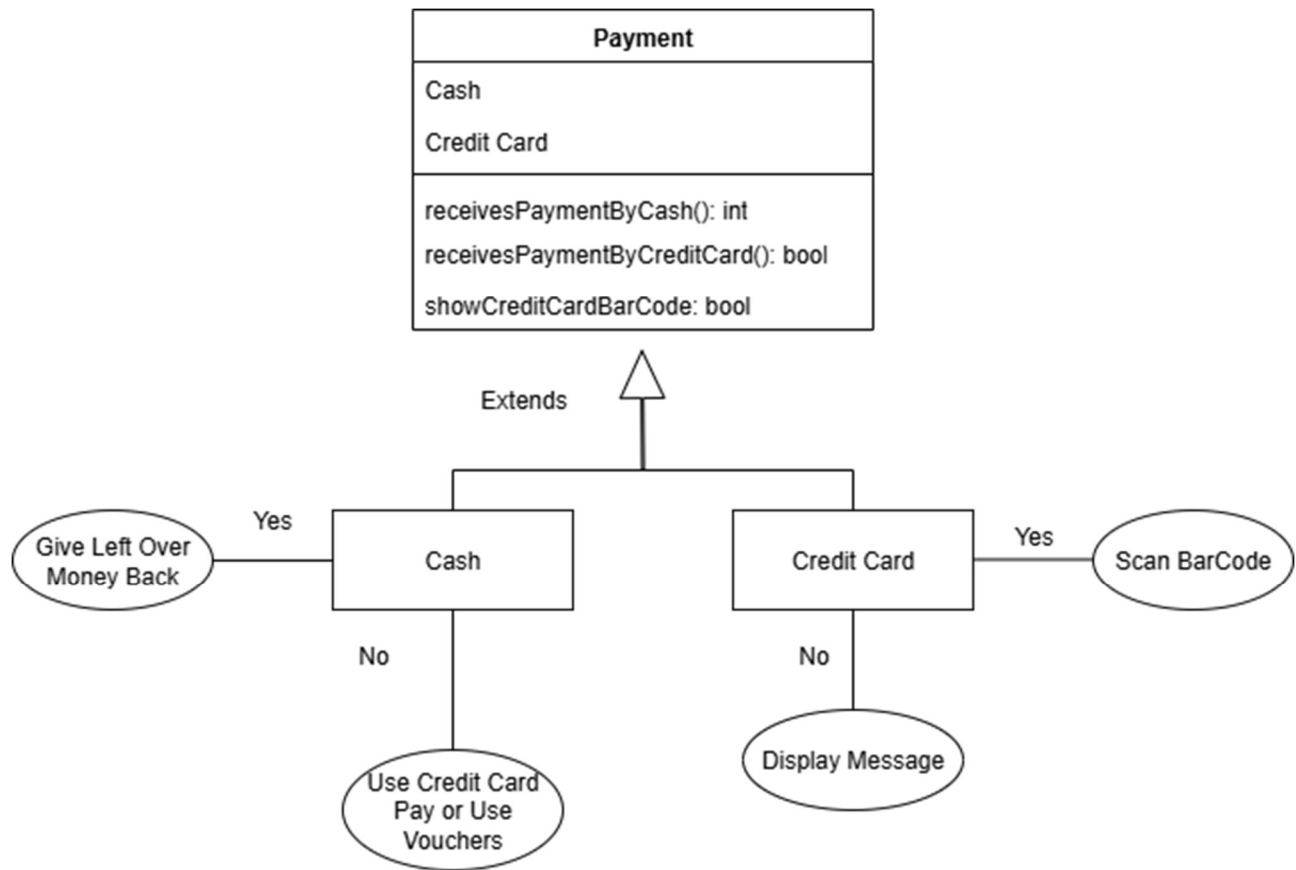
Menu Items



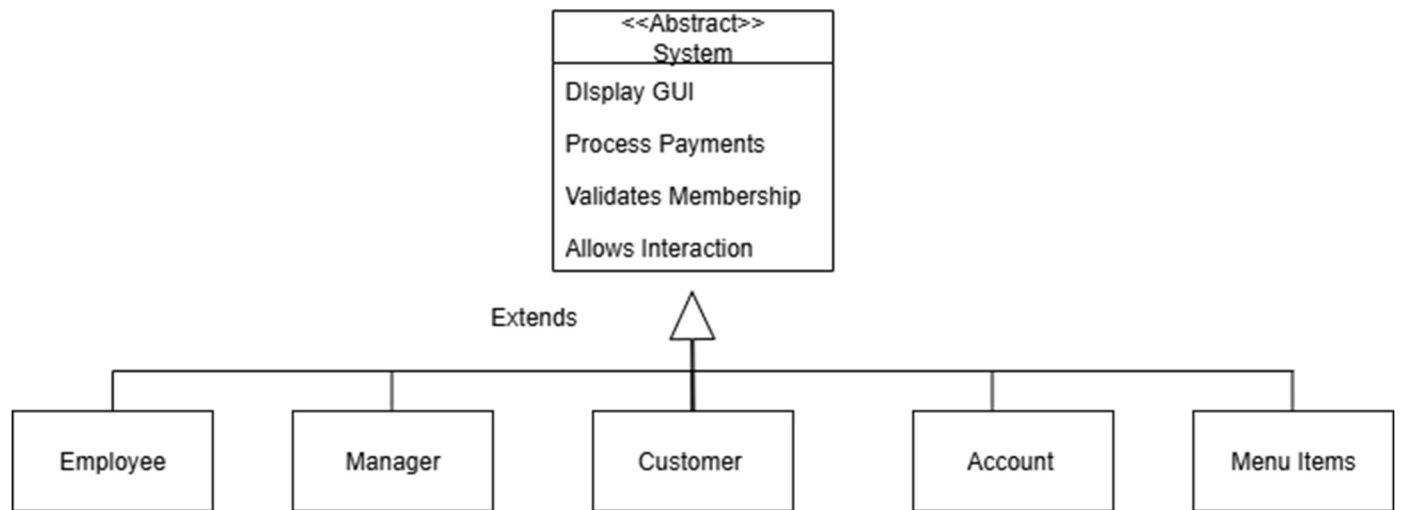
Customer Class



Payment Class



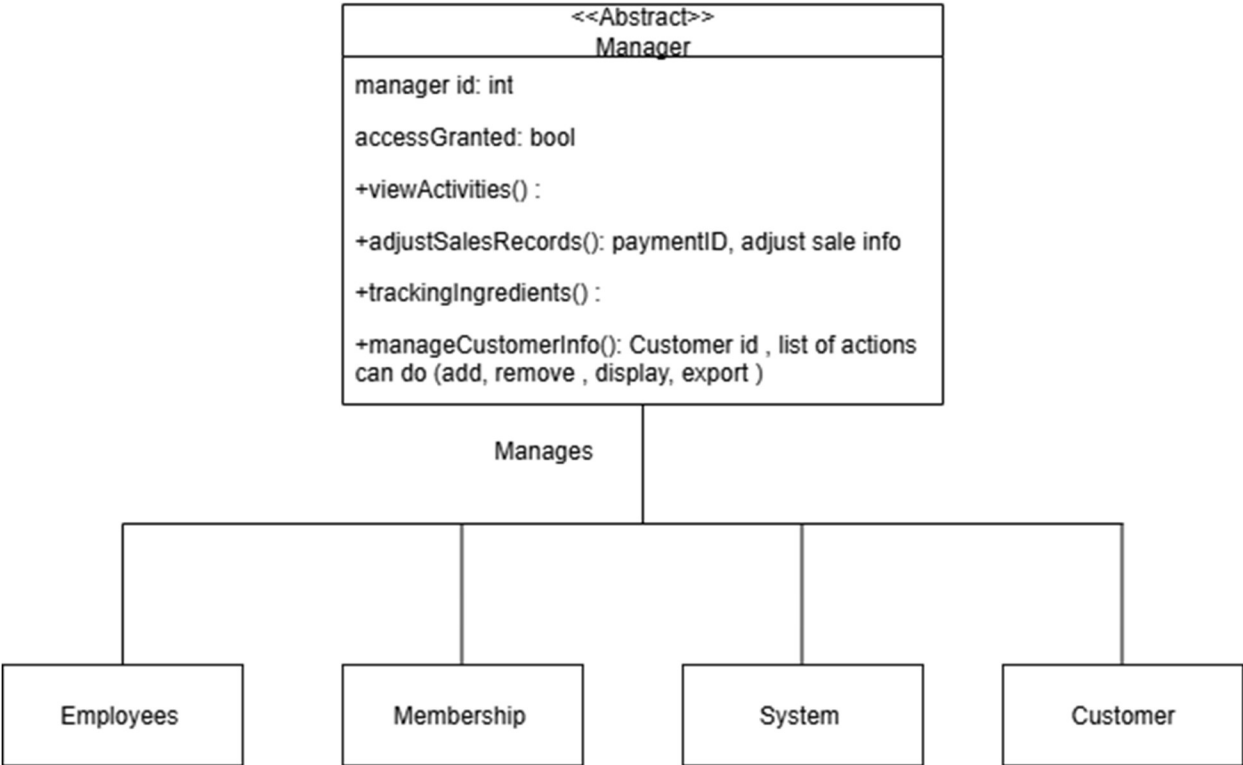
System Class



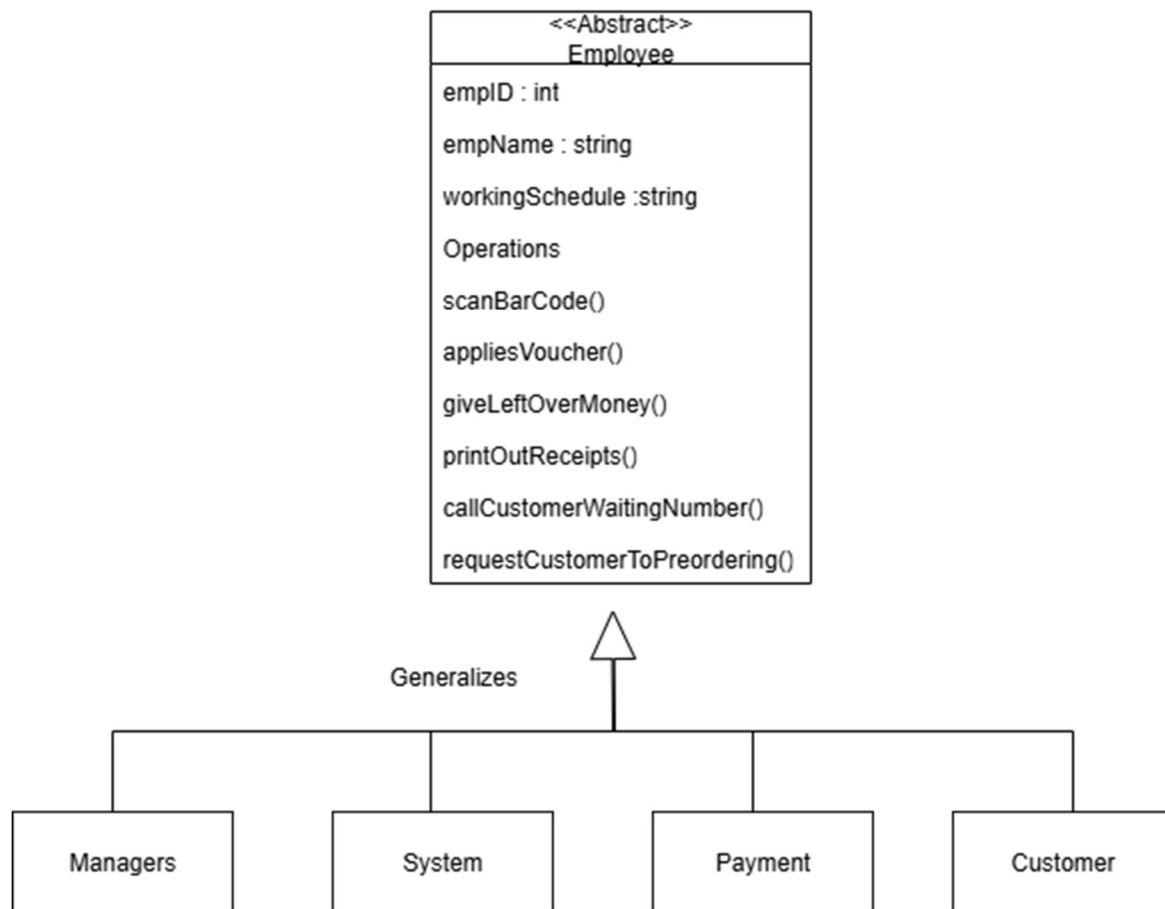
Products Class

Products
food items: id, string, int
drink items: id, string, int
getProductItems(): bool

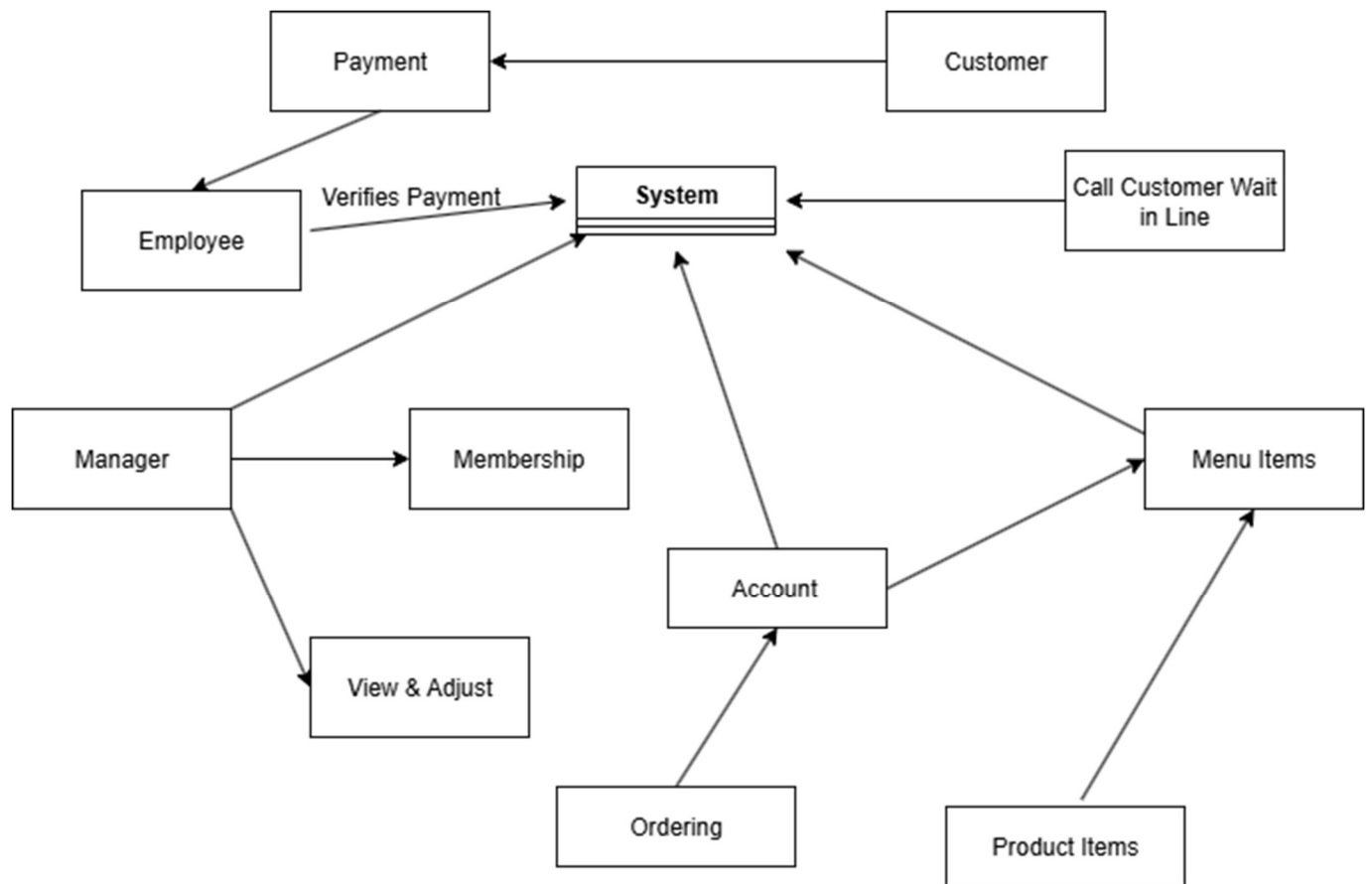
Manager Class



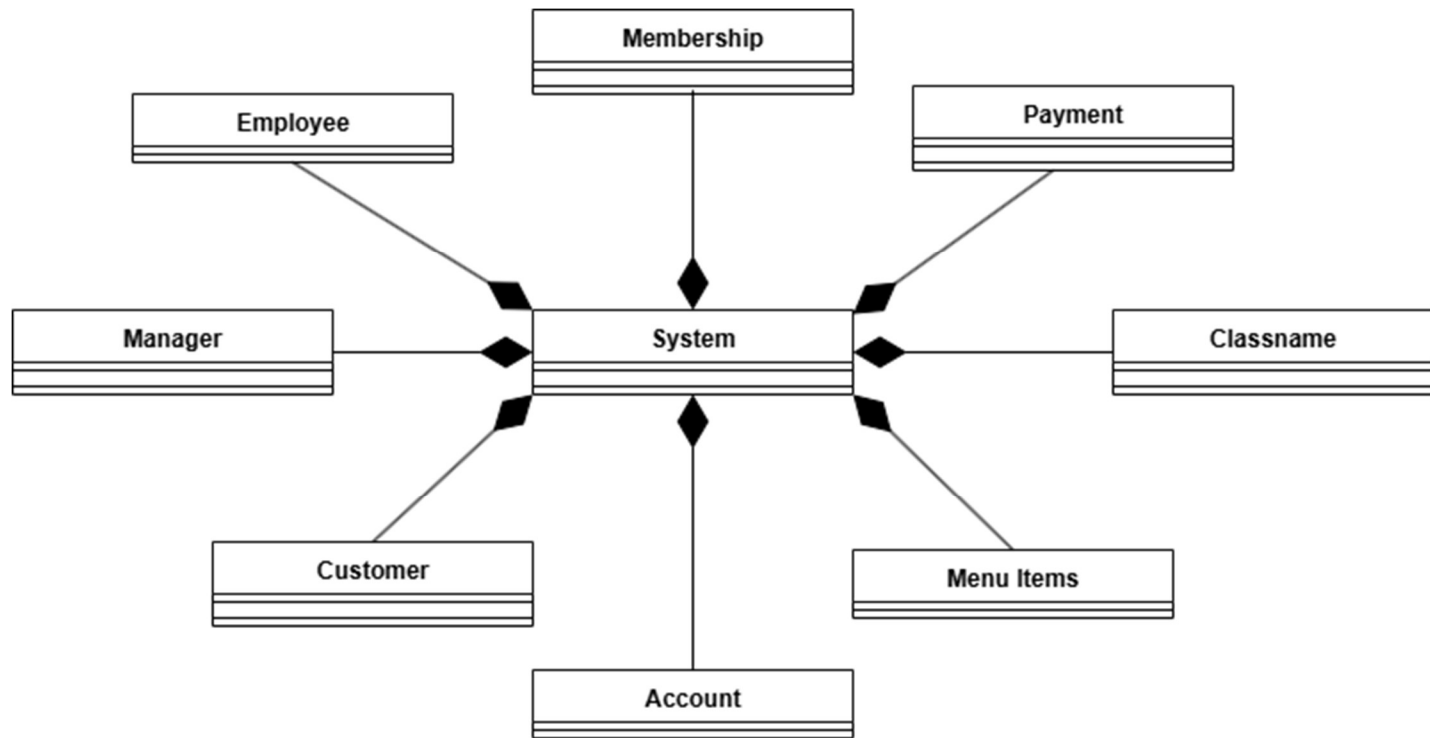
Employee Class



Graph of entities, attributes and showing relationship

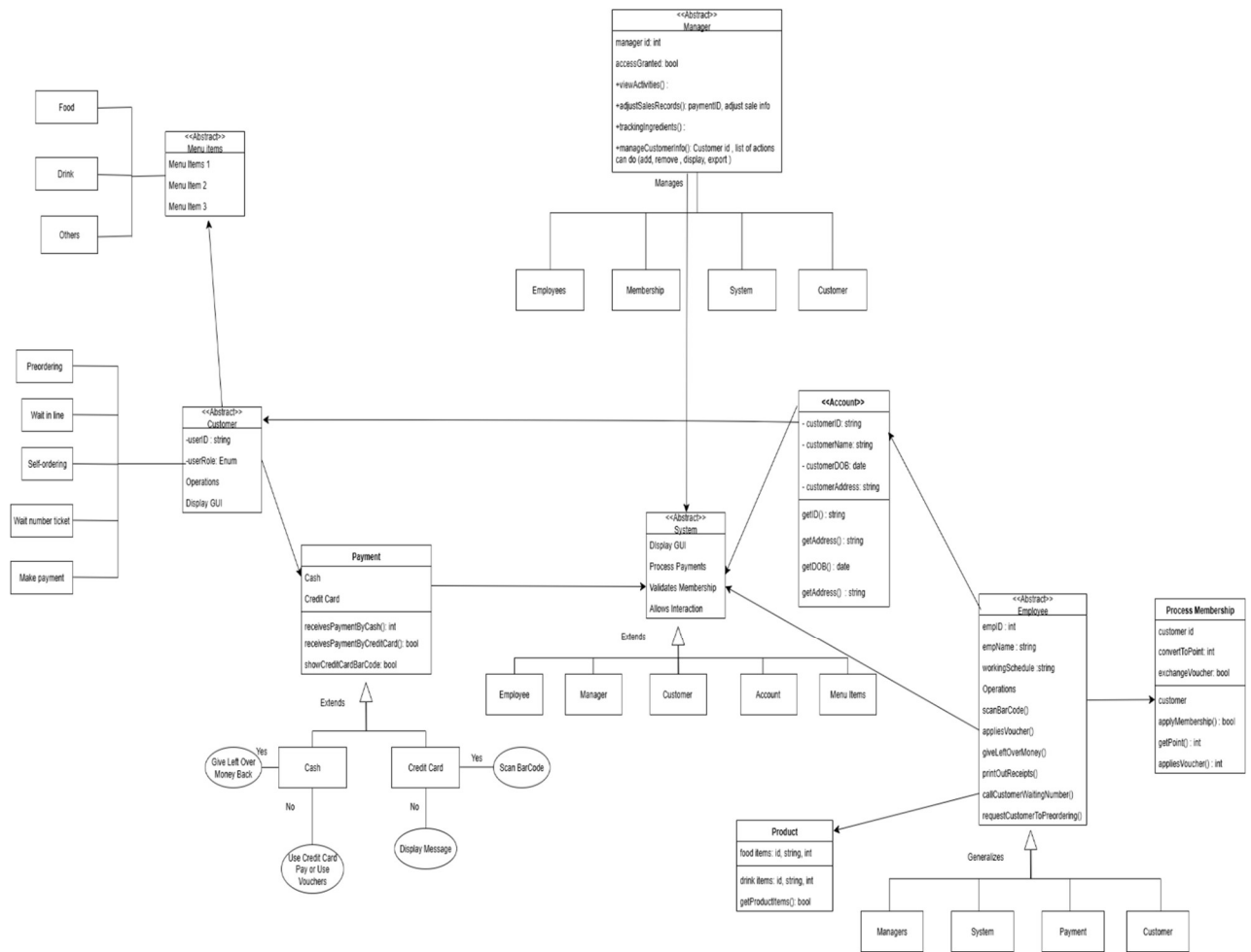


System Composition



Brief Sketch of Class Diagram

Notes: I really enjoy the process of drawing out class diagrams, and the relationship between attributes and custom data types. Very nice and beautiful (:

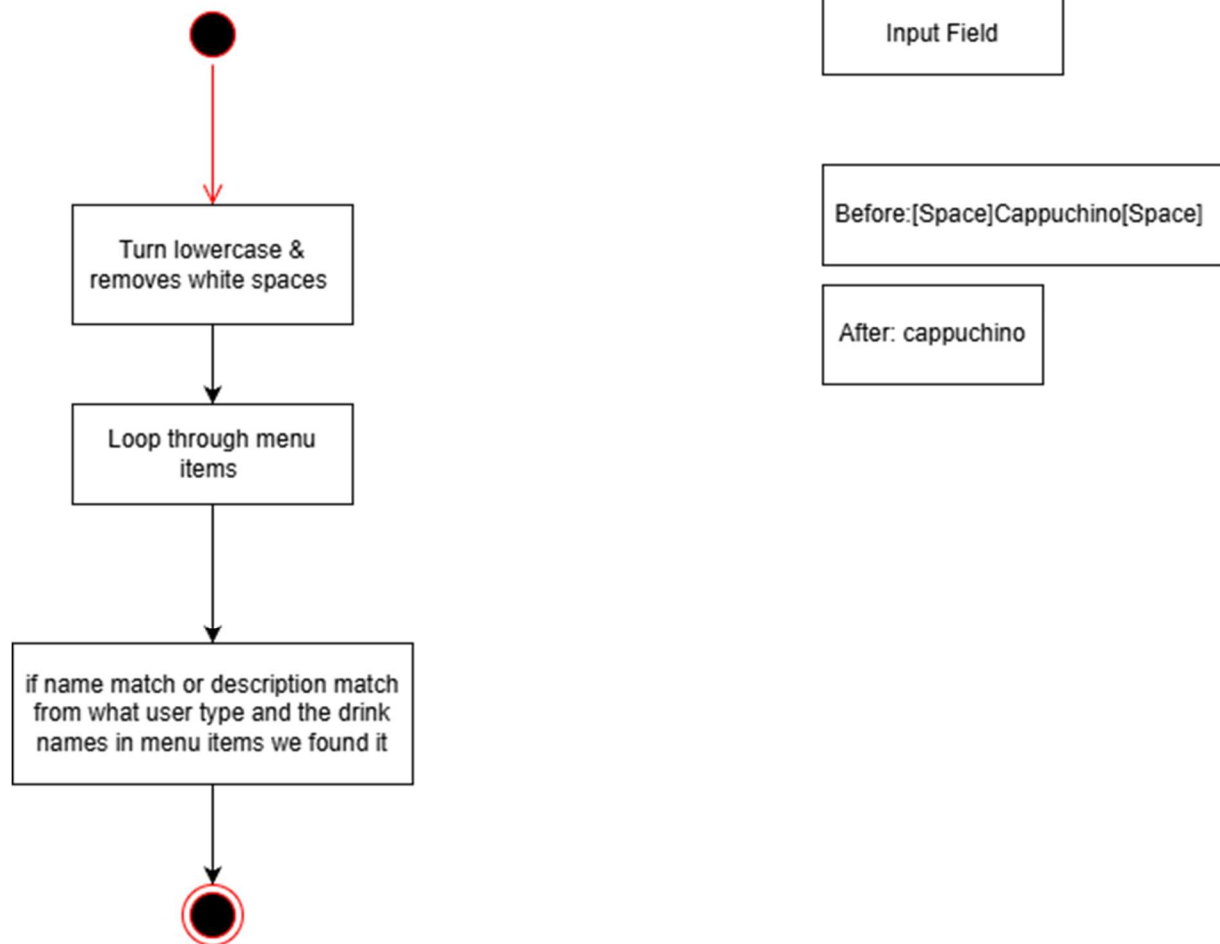


Algorithms and Data Structures

Explain the coding process

Describe the algorithms that implement mathematical models from your previous reports.

Algorithm activity diagrams



Complexity: $O(n * m)$

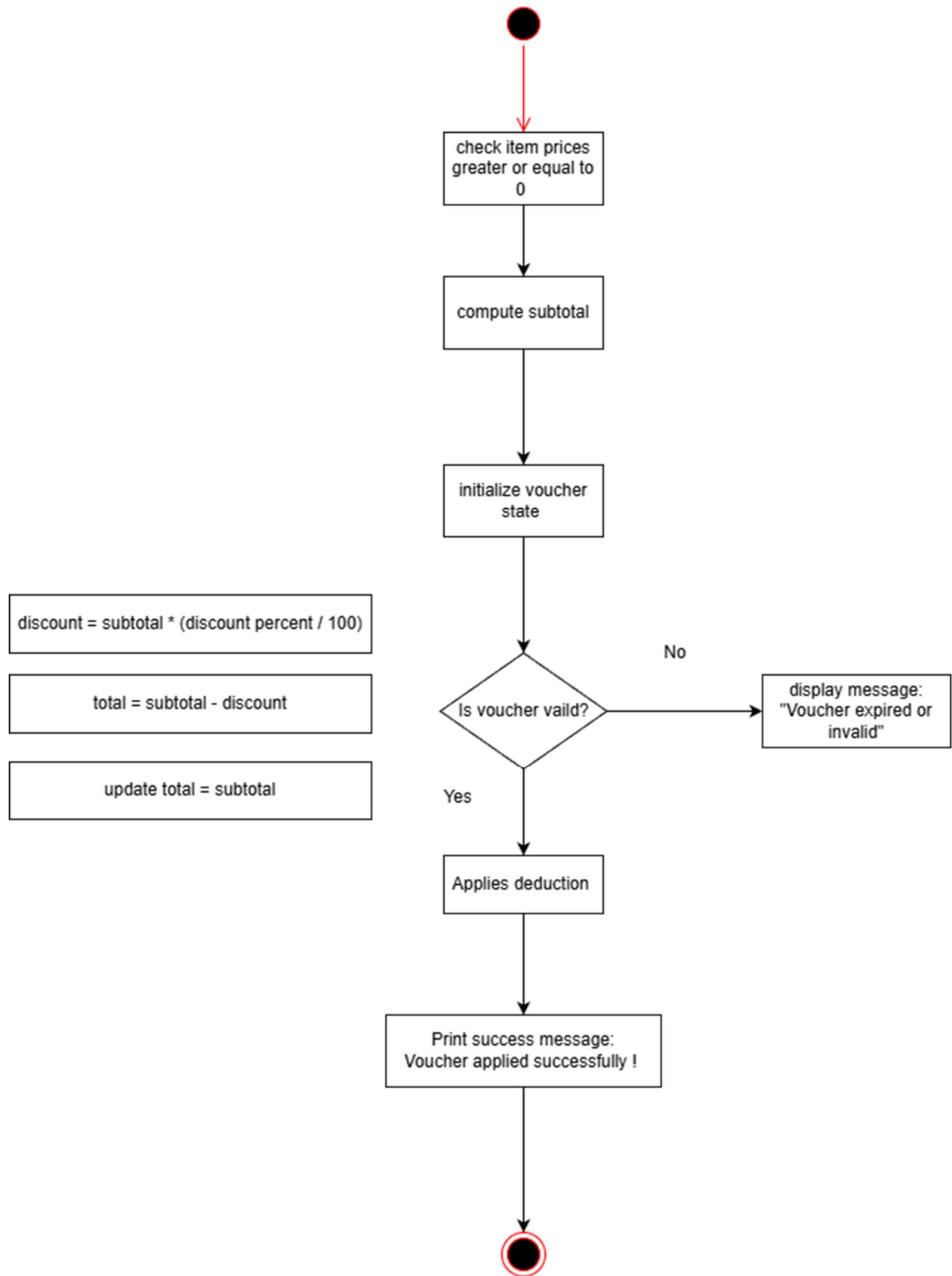
n is where the total number of items listed in the menu

For example: we have 11 drinks list in the menu, n will be 11

m : represent the total characters of the user type in the input field

for example: cappuchino = 10

$$O(n*m) = 11 * 10 = 110$$



Does your system use any complex data structures, such as arrays, linked list, hash tables, or trees?

Arrays

In my coding process I use arrays including in menu Items, cart and to filter item.

I rely on time complexity and space complexity. In this case the array operations I use perform $O(n)$. One reason why I choose arrays to do implements this is because arrays provide fast indexing to maintain and reusable structure for storing data efficiently. So, I would need to initialize the arrays, use the algorithm to process storing from menu Items.

I believe using a linked list would be more robust and useful than Arrays in my case, because the arrays are in $O(n)$ run time, if I implement a doubly linked list, I can use insertion at front and deletion at end to make it $O(1)$ run time. A better data structure that not only works but is reliable for this specific type of problem.

Hash Tables $O(1)$

I chose Hash table because it gives faster look up, I plan to use it in back end, for functionality like getting

Linked list $O(1) \sim$ head and tail

I chose linked list because it is going to be used in insertion and order deletion for individual order transactions

Fibonacci heaps $O(1)$

I chose Fibonacci heaps because it prioritizes and optimizes scheduling, and is efficient in storing data

Queue $O(1)$

I chose Queue for my system because I treat it as how often the user revisits the system and how long does it take for the user to complete the service.

Discussion: for the metric that I use to measure the effectiveness of complex data structures, I choose space complexity and average time complexity to see how well it works in real time.

For performance check and how to measure it technically, what I would do is find the execution time – how long does it takes for Queue, Fibonacci heaps, linked list, hash tables and arrays completes it task by taking the time after executing complex data structure minus time before we begin to execute it.

Traceability Matrix

No.	Priority Weight (1-5) 1:low , 5: high	Description
REQ1	5	Membership Points: turn vouchers into
REQ2	5	Activates self-service mode
REQ3	5	Payment features, user will provide cash / credit that will help them pay for the drinks
REQ4	5	Allows managers to edit records, items viewing and manages user information
REC5	3	The system should be handling for multiple launches of the menu shopping interface
REC6	3	Navigation to help user move from sections to sections and finish ordering process.
REC7	2	Customers should feel the system is Cleanly present and organize, no overcomplication
REC8	1	Upon using the coffee management system, all requires procedure and complex calculation and storing should not insist on the GUI but must strictly runs on backend and data base model.

Use case:

No.	Description
UC1	Membership points are to obtain points from buying products and customers can use it for future shopping
UC2	Self-service is to let customers order on their own, select items on menu, interacts with items and shows real time update with interactive features.
UC3	Payment service is a section that allows customers to pay through credit card or cash
UC4	Manager view / Sales adjustment is a section that allows manager to view / adjust sales record, track ingredients and managing customer information
UC5	Multi tabs, system to opens 30 screens that is stabilize, ensure responding and no unresponsive
UC6	To back navigation and quit the program
UC7	Easy to use functionalities with customer in mind design
UC8	Removing complex operations and formal, technical formulas on client side

Traceability Matrix

Requirement	PW	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8
REQ 1	5	X							
REQ 2	5		X						
REQ 3	5	X		X					
REQ 4	5	X			X	X			
REQ 5	3	X				X		X	
REQ 6	3	X					X		
REQ 7	2	X						X	
REQ 8	1	X						X	X
Max prior weight		5	5	5	5	5	3	3	1
Total prior weights		24	5	5	5	8	3	6	1